{EPITECH}

BLACK-BOX REPORT

REVERSE BINARY

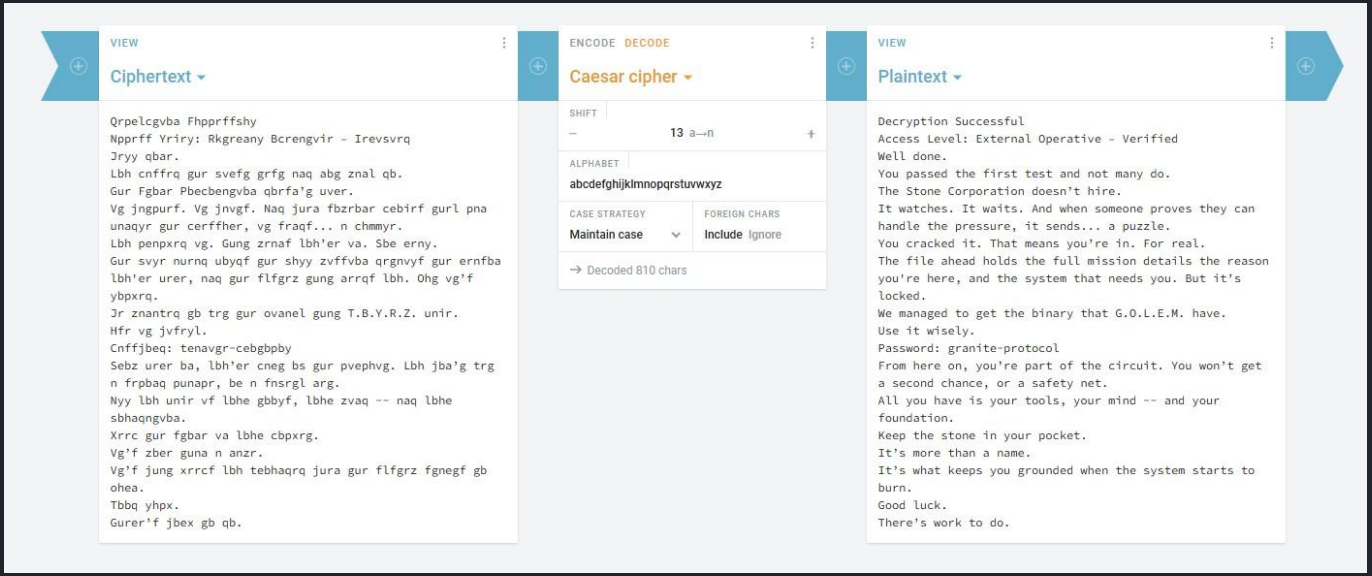


REPORT

Black-box Audit

In this phase, we analyzed the binary using reverse engineering tools and manual fuzzing.

List of the tools you used:
- strings
- upx
- caesar-cipher (url: https://cryptii.com/pipes/caesar-cipher/)



Key discoveries:
- identified that the binary was packed with UPX.
- discovered credentials (password: admin123).
- discovered vulnerability in the .obsidianrc, allowing for command injection.
- found that the application leaks data.

Vulnerability Report

Below is the list of vulnerabilities discovered, ranked by severity:

Vulnerability #1: System compromise

Severity: Critical
Type: Privilege Escalation / Bypass
Location: activate_emergency_protocols
Discovered in: Black-box

Description:
The application hid sensitive information using a binary packing (UPX). Using the utilitary upx, this was easily bypassed. Once the binary was unpacked, the application's internal strings revealed multiple passwords and flags used to bypass authentication for "Emergency protocols".

Proof of concept:

- detection of packing (analysis with strings failed to show readable logic, revealing a header)

```
5591    _j<X
5592    $Info: This file is packed with the UPX executable packer http://upx.sf.net $
5593    $Id: UPX 4.22 Copyright (C) 1996-2024 the UPX Team. All Rights Reserved. $
```

- filter with sensitive keywords

```
diana@Diana:~/cybersecurity/binaryV4$ strings obsidian | grep -iE "pass|admin"
Enter emergency password:
No password entered, emergency protocols not activated.
admin123
{Emergency protocols activated, you are now admin !}
- activate_emergency_protocols: Activate emergency protocols (requires admin).
Secret mode unlocked! Welcome, admin.
{ADMIN4242}
ThisIsTheBestPassword
{Correct password! Welcome, admin.}
Please enter your password:
```

- exploitation

```
obsidian> activate_emergency_protocols
Enter emergency password: admin123
{Emergency protocols activated, you are now admin !}
obsidian>
```

Impact:
An attacker can gain admin privileges over the application, managing to bypass all security layers, granting unauthorized access to multiple functions.

Vulnerabity #2: Command Injection via .obsidianrc

Severity: Critical
Type: Command Injection
Location: .obsidianrc
Discovered in: Black-box

Description:
The application includes a feature that allows users to create a configuration file named .obsidianrc. According to the application's own help command, this file is used to "automatically execute command at startup" using the syntax "exec <command>".
The application doesn't restrict which commands can be used.

Proof of Concept:

- output from the help command:

```
Configuration:
- .obsidianrc: Create this file in your home directory or current directory
  to automatically execute commands at startup.
  Use 'exec <command>' to execute shell commands.
```

```
diana@Diana:~/cybersecurity/binaryV4$ echo "exec cat /etc/hostname" > .obsidianrc
diana@Diana:~/cybersecurity/binaryV4$ ./obsidian
Diana
obsidian>
```

Impact:
This is a critical vulnerability because the application runs any command the user puts in the .obsidianrc file, which is why an attacker is able to execute malicious commands on the system. Since these commands run automatically as soon as the program starts, an attacker can take full control of the computer the moment the application is launched.

Vulnerability #3: Integer Overflow

Severity: Critical
Type: Integer Overflow; Improper Input Validation
Location: obsidian_lib.so, turbine_temperature
Discovered in: Black-box

Description:
The function does not validate the upper bounds of the temperature change input. The vulnerability comes from the use of a 32-bit signed integer variable to store the temperature. In C, a signed 32-bit integer has a maximum value of $2^{31} - 1$, which is 2.147.483.647. The program adds the provided number to the current temperature (20°C), thus exceeding the storage capacity of the variable. The most significant bit (the sign bit) is flipped and the number is interpreted as negative. The solution is to implement a check for a maximum input threshold before performing the sum.

Proof of Concept:

1. Run ./obisidian
2. Execute init_steam_turbine
3. Execute turbine_temperature
4. Input 2147483647

```
ana@Ana:/mnt/c/Users/ana/OneDrive - Universitatea Politehnica Bucuresti/Documents/Epitech/introduction_to_cybersecurity/obsidian$ ./obsidian
obsidian> init_steam_turbine
Start of the steam turbine initialized
Steam turbine status: OK
Steam valves status: OPEN
Pressurized water tank: OK
Pressurized water valves: READY
Steam turbine rotor status: OK
Starting steam turbine rotor...
Steam turbine rotor status: OK
Steam turbine rotor current speed: 1000 r/min
Reactor initialization complete.
obsidian> turbine_temperature
Enter the number of degrees you want to increase or decrease the turbine temperature : 2147483647
Turbine temperature is 20 degrees.
Turbine temperature is increasing : -2147483629
obsidian>
```

Impact:
- Logic manipulation: unintended negative values can cause unwanted events that affect the operation (in this case the way the turbines work will be altered).
- Safety checks: these kinds of systems usually have a threshold for high temperatures that cause the equipment to stop. An attacker could keep the system running at extremely high temperatures, while the program registers them as negative degrees and preventing the automatic emergency shutdown for that kind of temperature.

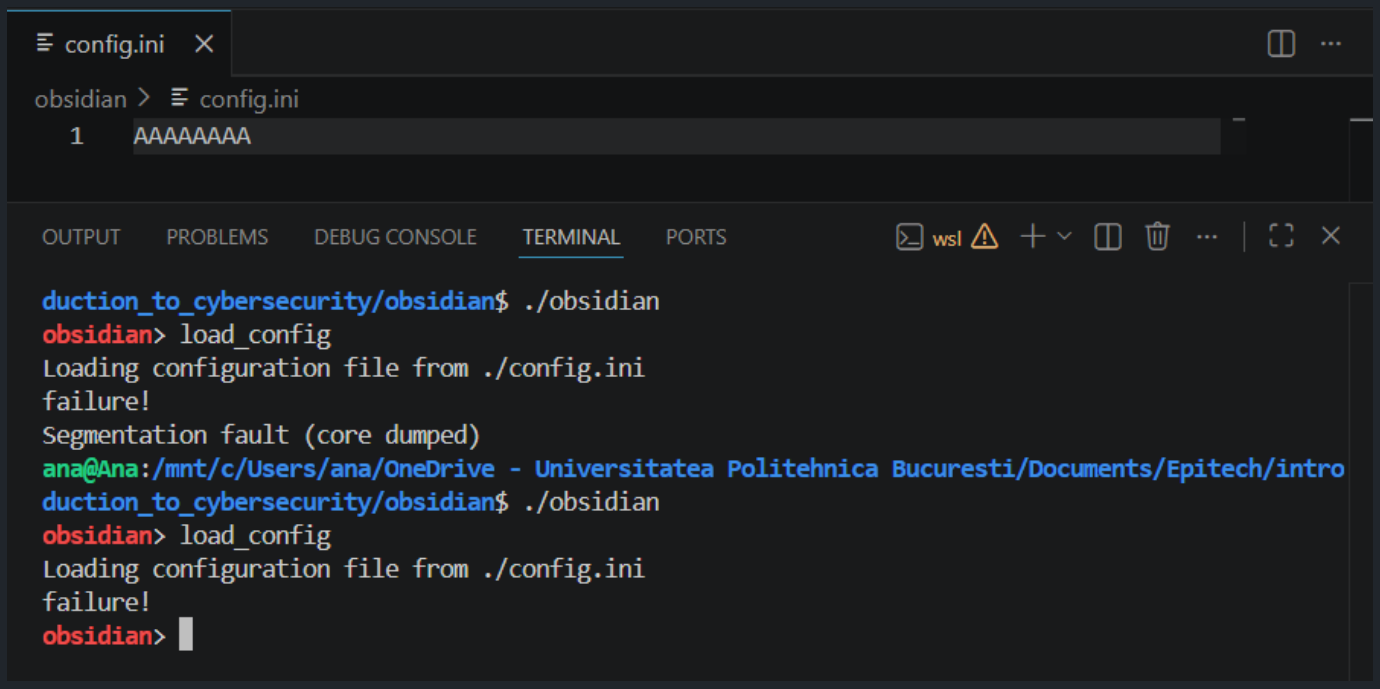
Vulnerability #4: Buffer Overflow

Severity: Critical
Type: Stack-based Buffer Overflow
Location: load_config
Discovered in: Black-box

Description:
The function does not check bounds when reading from the config.ini file. Providing a string longer than 12 characters causes a segmentation fault, leading to memory corruption.

Proof of Concept:

1. Create a file named config.ini in the same folder as the binary
2. Run ./obsidian
3. Execute load_config



*Note that the first example in the screenshot is for a 13-character string, while the second one is for the string that's displayed on the screen

Impact:
- Remote code execution: an attacker can overwrite the return address to redirect the program's execution flow to malicious code.

Vulnerability #5: Integer Overflow

Severity: Critical
Type: Integer Overflow; Improper Input Validation
Location: obsidian_lib.so, turbine_temperature
Discovered in: Black-box

Description:
The function does not validate the upper bounds of the temperature change input. The vulnerability comes from the use of a 32-bit signed integer variable to store the temperature. In C, a signed 32-bit integer has a maximum value of $2^{31} - 1$, which is 2.147.483.647. The program adds the provided number to the current temperature (20°C), thus exceeding the storage capacity of the variable. The most significant bit (the sign bit) is flipped and the number is interpreted as negative. The solution is to implement a check for a maximum input threshold before performing the sum.

Proof of Concept:

5. Run ./obisidian
6. Execute init_steam_turbine
7. Execute turbine_temperature
8. Input 2147483647

```
ana@Ana:/mnt/c/Users/ana/OneDrive - Universitatea Politehnica Bucuresti/Documents/Epitech/introduction_to_cybersecurity/obsidian$ ./obsidian
obsidian> init_steam_turbine
Start of the steam turbine initialized
Steam turbine status: OK
Steam valves status: OPEN
Pressurized water tank: OK
Pressurized water valves: READY
Steam turbine rotor status: OK
Starting steam turbine rotor...
Steam turbine rotor status: OK
Steam turbine rotor current speed: 1000 r/min
Reactor initialization complete.
obsidian> turbine_temperature
Enter the number of degrees you want to increase or decrease the turbine temperature : 2147483647
Turbine temperature is 20 degrees.
Turbine temperature is increasing : -2147483629
obsidian>
```

Impact:

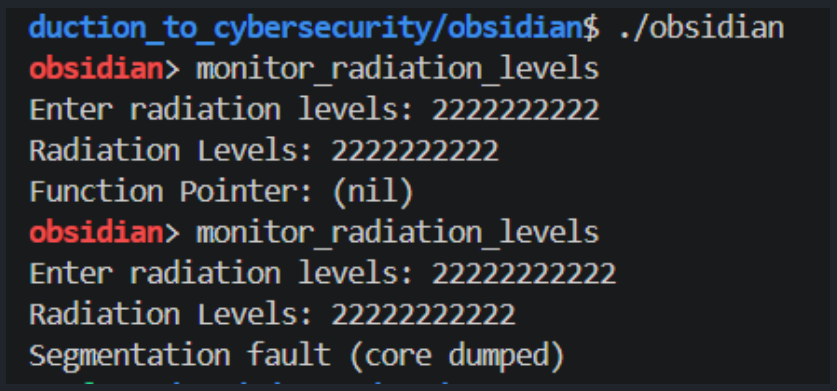
- Logic manipulation: unintended negative values can cause unwanted events that affect the operation (in this case the way the turbines work will be altered).
- Safety checks: these kinds of systems usually have a threshold for high temperatures that cause the equipment to stop. An attacker could keep the system running at extremely high temperatures, while the program registers them as negative degrees and preventing the automatic emergency shutdown for that kind of temperature.

Vulnerability #6: Buffer Overflow

Severity: Critical
Type: Stack-based Buffer Overflow; Function Pointer Hijacking
Location: obsidian, monitor_radiation_levels
Discovered in: Black-box

Description:
The function does not validate user input size before storing it in memory. Providing a string longer than 10 characters causes a segmentation fault. The function allocates a buffer on the stack for this radiation level string and, immediately after, there is a function pointer used by the system. By inputting 11 characters, it overflows its boundaries by overwriting into the first byte of the function pointer.

Proof of Concept:
1. Run ./obsidian
2. Execute monitor_radiation_levels
3. Enter a 11-character number for radiation levels



*Note that the first example in the screenshot is for a 10-character string, while the second one is for an 11-character string

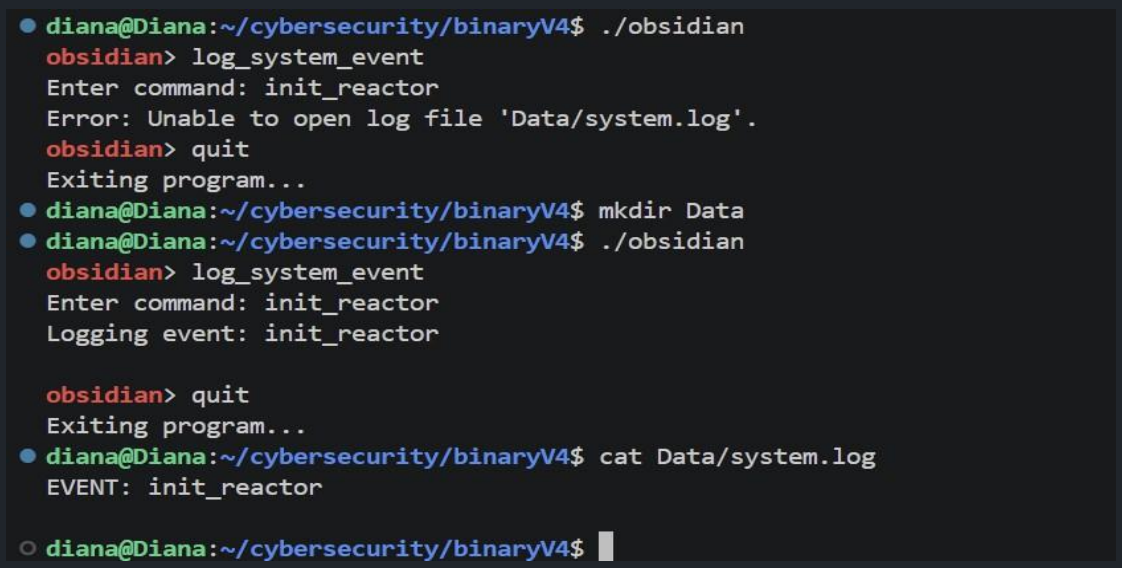
Impact:
- Control Flow Hijacking: an attacker can replace the pointer with the memory address of a malicious function or one that requires admin privileges.

Vulnerability #7: Information Disclosure via Error Messages

Severity: High
Type: Information Disclosure
Location: log_system_event
Discovered in: Black-box

Description:
The application leaks sensitive path information by outputting direct system error messages when failing to access files. Specifically, when the directory Data is missing, the application reveals the location of its log file (Data/system.log). Because the program doesn't use generic error messages, the application allows any user to see the internal folder structure that the program expects to find.

Proof of Concept:



Impact:
By revealing the exact name and location of the log files, the application provides the attacker with essential reconnaissance data. An attacker could attempt to overwrite system logs, making the application unsafe. Moreover, if an attacker does something bad in the system, they can simply delete or modify the file, making it impossible for an admin to see what happened.

Vulnerability #8: Insecure Log Handling

Severity: High
Type: Log injection
Location: read_turbine_config
Discovered in: Black-box

Description:
The application leaks path information regarding the folder. By leveraging the path leakage identified in vulnerability #3, an attacker knows the application writes to and reads from Data/system.log. The attacker can manually insert keywords into system.log, tricking the application into executing those strings as internal events if the log file is loaded as a config file.

Proof of Concept:

```
obsidian> read_turbine_config
Enter the configuration file name: d
Error: Unable to open file: Data/d
obsidian> read_turbine_config
Enter the configuration file name: system.log
Reading configuration file: Data/system.log
EVENT: ls

obsidian>
```

```
obsidian> read_turbine_config
Enter the configuration file name: system.log
Reading configuration file: Data/system.log
EVENT: ls

obsidian> quit
Exiting program...
diana@Diana:~/cybersecurity/binaryV4$ echo "EVENT: ls -l" >> Data/system.log
diana@Diana:~/cybersecurity/binaryV4$ ./obsidian
obsidian> read_turbine_config
Enter the configuration file name: system.log
Reading configuration file: Data/system.log
EVENT: ls

EVENT: ls -l
obsidian>
```

Impact:
This allows an attacker to manipulate the application's logic. By manipulating the system.log file that the program later uses, an attacker can maybe later trigger unintended events.

Vulnerability #9: Boundary Logic Flaw

Severity: Medium
Type: Improper Input Validation
Location: obsidian, set_reactor_power
Discovered in: Black-box

Description:
The function is not consistent in validating user input for the reactor power. It recognizes that, if the provided number is smaller than -1000, the power is set to an invalid value. However, allowing the user to actually introduce such a number leads to a meltdown, which would otherwise only be made possible by an admin.

Proof of Concept:
1. Run ./obsidian
2. Execute set_reactor_power
3. Enter a number smaller than -1000

```
obsidian> set_reactor_power
Enter reactor power level: -1000
Reactor power set to: 0
Reactor operating within safe parameters.
obsidian> set_reactor_power
Enter reactor power level: -1001
Reactor power set to an invalid value: -1
Reactor core meltdown imminent!
```

Impact:
- Privilege escalation: an attacker could possibly trigger a dangerous state, such as the meltdown.
- Denial of service: after the meltdown is initiated, the system becomes unusable.

v 1.0

